



# Voiceover script

For the 10:10 walkthrough recording. What to say, segment by segment, in the order the video moves.

---

Eleven segments, **1375 words**. At a normal voiceover pace that is about **9 minutes 29 seconds** of speech inside a **10:10** recording, which leaves roughly **41 seconds** of pause spread across the eleven breaks. That is the right shape for narration over a screen recording: it should breathe, not race.

**1375**

words to read

**9.5 min**

of speech at 145 wpm

**41 s**

of pause to spend

## How to use this

I have not seen your recording — I cannot watch video — so the timecodes below are where the *words* divide, not where your cursor moves. Treat them as a running order, not as cues to hit.

- **Record segment by segment, not in one take.** Eleven short recordings are far easier to fix than one long one, and they let you slide each segment to where its part of the screen actually starts.
- **The word count is the number that matters.** Each segment is sized for its slot; if your take spends longer somewhere, the pause budget absorbs it.
- **Read the paragraph breaks as pauses.** They are placed where a viewer needs a moment to look at what you just pointed at.
- **If a segment does not match your footage,** cut it whole rather than trimming sentences — each one is self-contained and the argument survives losing any single middle segment.
- **Tell me the timestamp each section actually starts at** and I will re-time this exactly, rather than proposing boundaries.

## The running order

| Starts | Segment                                         | Words | Speech |
|--------|-------------------------------------------------|-------|--------|
| 0:00   | Who, and the problem this exists for            | 145   | 60s    |
| 1:03   | Settings, and what kind of thing this is        | 153   | 63s    |
| 2:10   | Overview: the two charts, and how to check them | 211   | 87s    |
| 3:41   | Decision record: the reasoning, not the row     | 172   | 71s    |
| 4:56   | Incident replay: written at the time            | 98    | 41s    |
| 5:40   | Recovery: gross, fees, net                      | 133   | 55s    |
| 6:38   | Actions: bounded, and always reversible         | 67    | 28s    |
| 7:10   | Refused: the half a dashboard never shows       | 114   | 47s    |
| 8:01   | Rollbacks: the system correcting itself         | 53    | 22s    |
| 8:26   | Audit ledger: checkable outside the tool        | 73    | 30s    |
| 9:00   | How it was measured, and where the AI is        | 156   | 65s    |

The four widest segments are the introduction, the settings page, the overview and the decision record — between them they are the argument. Everything after the decision record is evidence for it, and each of those can be shortened without losing the thread.

## The narration

Read each block at a normal pace and leave the pauses where the paragraphs break. Bold is a number — land on it. Square brackets are notes to you, not words to read.

0:00

Who, and the problem this exists for · 145 words · ~60s

ON SCREEN The board. Logo, the two-line statement, the four quotes.

Hi, I'm A Sachin Kumar. This is RazorGuard, my submission for Track 03, AI Revenue Recovery.

Start with the problem, because it is why the whole thing is shaped this way. A payment gateway is not one pipe. Traffic splits across acquiring banks, methods and issuing banks, so one stream is really a few dozen narrow routes. And when payments fail, it is almost never everywhere at once. It is one acquirer, or one issuing bank, or one method on one acquirer.

Say a route carrying four percent of volume collapses from ninety-six percent success to forty. Your headline success rate moves **two points** — inside normal daily variation. No alert fires, nobody is paged, and the money leaves quietly until somebody notices by hand.

RazorGuard watches every route, catches that, moves traffic around it under written bounds — and then proves what the moving was worth.

**1:03****Settings, and what kind of thing this is · 153 words · ~63s****ON SCREEN** The Settings page. The world block, then the policy bounds.

First the settings, because this page says what kind of claim I am making.

This is a simulated payment fleet. Three acquiring gateways, three methods, eight issuing banks — seventy-two routes. Nine hundred payments a minute on a daily traffic curve, and **eighteen** failures injected across **2 simulated days**. The world is invented. The measurement is not, and I will come back to exactly what that means.

The seed drives every random draw — traffic, outcomes, and where the failures land. Both arms always share it.

These are the policy bounds. How much traffic one action may move. How far a route may drift from its configured weights. How much evidence a detection needs before it may move money at all.

None of it is decoration — every control feeds the real simulator and the real policy engine, and changing one re-runs both arms. One of these numbers overturned a decision I had made on instinct.

**2:10****Overview: the two charts, and how to check them · 211 words · ~87s****ON SCREEN** Overview. The cumulative chart, then the success-rate chart, then the chart's own  menu.

This is the overview. Two charts, because they answer two different questions.

The top one is cumulative recovery. Every step up is one payment that failed with routing off and succeeded with it on. The red bands are the injected failures — the line climbs inside them and flattens between, which is what you would expect if the system is doing anything at all. Over the full run it reaches **11,452** payments.

The second is the success rate with the router on and off, both arms on one axis. The lines sit close, and that is the point — the gap is the story. Half a percentage point is worth crores at this volume. Green triangles are routing actions; red ones are rollbacks.

Both zoom. Scroll into a single incident, drag to pan, hover for the values at that exact minute.

And they are not pictures. Every chart carries this menu — save as PNG or SVG, or view source, which gives you the chart specification and the data behind it. Nothing here has to be taken on trust; you can export it and check it elsewhere.

And clicking any point on the timeline opens that minute — the numbers either side, which incident was live, and every ledger entry within three minutes of it.

If the click-a-minute panel is not in your take, drop that last paragraph and give the four seconds to the overview segment.

**3:41**      **Decision record: the reasoning, not the row** · 172 words · ~71s

ON SCREEN    Decision record. The board of eighteen, then the top card.

This is the decision record, and it is the centre of the project.

Eighteen incidents, ranked by the money the control arm actually lost to them — not by how loud they were. The table gives you the shape of the run in one screen. Below it, each becomes a card carrying its reasoning, because a table can tell you an incident happened and not why the system did what it did.

This is the worst one. A gateway hard outage across **24 routes**. Success rate on those routes fell from **93.7%** to **33.4%**. It was caught in **one minute**, with **Rs 64.74 L** at risk.

This row is the decision path. The signal. What broke. What it was measured over. Time to detect. Money at risk. The routes it moved. And at the end, what it did.

The cells tinted blue were measured against the control arm during the run. The plain ones are policy written before it started. Two very different kinds of claim, so they do not look the same.

**4:56**      **Incident replay: written at the time** · 98 words · ~41s

ON SCREEN    Incident replay, with the ledger lines scrolling.

Incident replay is that same incident as a ledger, line by line, exactly as the control plane wrote it. Nothing here is assembled afterwards.

Detection. Then a proposal. Then a decision. Then an action. And in among them, refusals — each with the rule that caused it.

One detail. Entries are matched to the incident by subject as well as by time, because eighteen incidents over two days overlap and filtering on time alone hands one incident another's actions. Before I fixed that, two issuer-wide faults reported themselves as routed around — when what they had correctly done was escalate.

**5 : 40**      **Recovery: gross, fees, net** · 133 words · ~55s

ON SCREEN    Recovery. The three-row table, then the per-incident bars.

Recovery breaks the money down in three rows rather than one.

Gross value recovered, **Rs 1.20 cr**. Then the processing fees on it — because those payments never settled in the control arm, so it never paid a fee on them. And then net, **Rs 1.19 cr**.

That matters in India specifically. UPI carries zero MDR by regulation, so a recovered UPI payment is pure margin; a recovered card payment gives one point eight percent of itself back. Here fees came to about in fees. Here the fees came to about **1%** of gross.

And below, where each rupee came from, incident by incident. The negative bars are incidents where the routing did not pay. They are shown rather than dropped, because a system that only reports its wins is not reporting a measurement.

**6 : 38**      **Actions: bounded, and always reversible** · 67 words · ~28s

ON SCREEN    Actions. The three panels, then the ledger.

Actions is every time it moved money — **100** of them across the run.

Each shifts at most eighty percent of the source gateway's share for one route, and always leaves a **three percent** canary behind. That canary is not politeness — without traffic staying on the failing route there is nothing to compare the destination against, and the system goes blind exactly when it needs to be watching.

**7:10**      **Refused: the half a dashboard never shows** · 114 words · ~47s

ON SCREEN    Refused. The reconciliation table, then the rules.

This is the page I would most want to be asked about.

**93** times the system declined to move money. And the top of the page reconciles that number instead of asserting it: **57** refusals that named a policy rule, **32** rollbacks, and **4** alarms covered by an already-open breaker. They add up.

Below that, every rule that fired, how often, and in plain English what that rule is protecting against.

The one I would point at is the efficacy breaker. It measures the realised effect of the system's own recent shifts, and when they stop paying, it stops making them. It is the rule that fires when the system is wrong about itself.

**8:01**      **Rollbacks: the system correcting itself** · 53 words · ~22s

ON SCREEN    Rollbacks. The ledger lines.

**32** times it moved traffic, measured the destination, found it worse than its own baseline, and put the traffic straight back.

A rollback is not a failure of the system. It is the system working. The failure would be moving traffic onto a degrading route and leaving it there because nothing was watching.

**8:26**      **Audit ledger: checkable outside the tool** · 73 words · ~30s

ON SCREEN    Audit ledger. Filter, search, then the CSV download.

The audit ledger is the complete record — **1,078 events**, append-only and sequence-numbered, written as the run proceeded rather than assembled for a demo.

Filterable by event kind, and searchable across the subject, the rule and the text. And it downloads as CSV — so every claim I have made in this video can be checked outside the tool entirely.

Every money movement and every refusal is in here, with the rule that decided it.

9:00

How it was measured, and where the AI is · 156 words · ~65s

ON SCREEN How it was measured. The provenance row, then the bottom row.

Last page, and the one that decides whether any of this counts — because a simulator can be made to say anything.

Three things stop it. The detectors cannot import the file that defines the incidents — a test fails if anyone tries. The recovery figure comes only from a paired control arm facing bit-identical demand. And if the arms ever diverge, the run refuses to report a figure at all.

Across **8 seeds: Rs 1.23 cr** recovered, ninety-five percent confidence interval **Rs 1.18 cr to Rs 1.28 cr**. That is **38.2%** of the money those incidents put at risk. **Zero of 8** seeds lost money.

And this last row is where the AI is. Detection, attribution, policy and routing are deterministic. A model writes the operator note, investigates a past decision and advises on escalations — and can change none of them. An LLM that can move money is a liability.

That is RazorGuard. Thank you for watching.

## Every number you say, and where it comes from

All of it is interpolated when this document is built — from `bench/results/*.json` and from the same control-plane run the console renders — so the narration and the screen cannot disagree.

| Said aloud                 | Value                    | Source              |
|----------------------------|--------------------------|---------------------|
| Recovered, mean of 8 seeds | Rs 1.23 cr               | validate.py         |
| 95% confidence interval    | Rs 1.18 cr to Rs 1.28 cr | the same            |
| Share of money at risk     | 38.2%                    | the same            |
| Seeds that lost money      | 0 of 8                   | the same            |
| This run, gross            | Rs 1.20 cr               | experiment.py       |
| This run, net of fees      | Rs 1.19 cr               | the same            |
| Fees as a share of gross   | 1.1%                     | the same            |
| Payments saved             | 11,452                   | the same            |
| Worst incident, before     | 93.7%                    | console run, seed 7 |
| Worst incident, during     | 33.4%                    | the same            |
| Worst incident, at risk    | Rs 64.74 L               | the same            |

Worst incident, routes

24

the same

## Two things not to say

- **Do not call the money real.** The world is simulated and the measurement is real; those are different sentences and the second one is the impressive one. If you blur them, a judge who notices stops believing the rest.
- **Do not quote the single-run figure as the result.** The tile says **Rs 1.19 cr** for this run; the defensible figure is **Rs 1.23 cr** across 8 seeds with an interval. The script says the second one.